

Математические основы алгоритмов, осень 2020 г.
 Лекция 5: Пути между всеми парами вершин в графе,
 алгоритм Варшалла, алгоритм Варшалла-Флойда.
 Использование умножения матриц. Алгоритм Штрассена
 быстрого умножения матриц. Умножение булевых матриц
 через числовые. Метод четырёх русских*

Александр Охотин

10 апреля 2021 г.

Содержание

1 Пути между всеми парами вершин	1
1.1 Очевидный алгоритм для транзитивного замыкания	2
1.2 Алгоритм Варшалла	2
1.3 Кратчайшие пути с весами	3
1.4 Обобщение до полуколец	4
1.5 Достижимость в графе и умножение матриц	5
2 Быстрое умножение матриц	5
2.1 Алгоритм Штрассена	5
2.2 Время работы рекурсивных алгоритмов	8
2.3 Умножение булевых матриц через числовые	10
3 Метод четырёх русских	10

1 Пути между всеми парами вершин

Дан ориентированный граф $G = (V, E)$, где $V = \{1, \dots, n\}$, а $E \subseteq V \times V$. Ставится задача проверить существование пути из каждой вершины в каждую — то есть, для каждой пары вершин (i, j) , определить, есть ли путь из i в j .

Эти сведения составляют новый граф с тем же множеством вершин, называемый *транзитивным замыканием графа* G : это граф $G^* = (V, E^*)$, где $(i, j) \in E^*$, если в G есть путь из i в j .

*Краткое содержание лекций, прочитанных студентам 1-го курса СПбГУ, обучающимся по программам «Математика» и «МАиАД», в осеннем семестре 2020–2021 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/algorithms_2020/.

1.1 Очевидный алгоритм для транзитивного замыкания

Очевидный алгоритм (алгоритм 1) будет перебирать все пары вершин (i, j) , а для каждой пары — все промежуточные вершины k , для которых есть дуги (i, k) и (k, j) . Если при этом дуги (i, j) ещё нет, она будет добавляться. Перебор всех пар (i, j) будет продолжаться, пока можно добавить новые дуги.

Алгоритм 1 Очевидный алгоритм для транзитивного замыкания

```
1: while можно добавить дуги do
2:   for  $i = 1$  to  $n$  do
3:     for  $j = 1$  to  $n$  do
4:       for  $k = 1$  to  $n$  do
5:         if  $(i, k) \in E \wedge (k, j) \in E$  then
6:           добавить дугу  $(i, j)$  к  $E$ 
```

Утверждение о правильности алгоритма — следующий инвариант внешнего цикла.

Лемма 1. После каждой t -й итерации внешнего цикла будут просчитаны все пути длины не более 2^t , и для каждого из них в граф будет добавлена дуга.

Это утверждение доказывается индукцией по t .

Отсюда следует, что время работы алгоритма — не более, чем $n^3 \log_2 n$ шагов.

1.2 Алгоритм Варшалла

Оказывается, что можно построить транзитивное замыкание за время n^3 , обойдясь без внешнего цикла («пока можно добавить дуги»). Для этого достаточно — внезапно — поменять циклы местами, сделав цикл по промежуточной вершине k внешним.

Алгоритм 2 Алгоритм Варшалла для транзитивного замыкания

```
1: for  $k = 1$  to  $n$  do
2:   for  $i = 1$  to  $n$  do
3:     for  $j = 1$  to  $n$  do
4:       if  $(i, k) \in E \wedge (k, j) \in E$  then
5:         добавить дугу  $(i, j)$  в  $E$ 
```

Время работы очевидно — ведь каждая тройка (i, k, j) рассматривается лишь однажды. Но почему алгоритм 2 работает правильно, то есть действительно строит транзитивное замыкание графа?

Лемма 2 (Варшалл [1962]). После k -й итерации внешнего цикла найдены все пути, проходящие только через промежуточные вершины из множества $\{1, \dots, k\}$ (как показано на рис. 1(левом)).

Доказательство. Доказывается индукцией по числу итераций.

Базис $k = 0$: путей нет, верно.

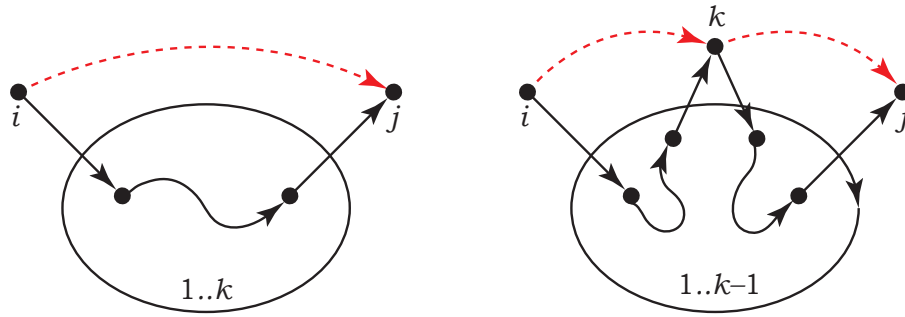


Рис. 1: (слева) Условие правильности алгоритма Варшалла; (справа) индукционный переход в доказательстве.

Переход. К началу k -й итерации все пути, проходящие через промежуточные вершины из $\{1, \dots, k-1\}$ уже найдены. Пусть кратчайший путь из i в j проходит только через вершины из $\{1, \dots, k\}$. Если через k он не проходит, то он уже построен. Если же он проходит через k , как изображено на рис. 1(правом), то он когда-то приходит в неё впервые — и, стало быть, есть путь из i в k , проходящий только через промежуточные вершины из $\{1, \dots, k-1\}$ — и когда-то в последний раз её покидает, так что есть путь из k и j , тоже проходящий только через $\{1, \dots, k-1\}$. Эти два пути уже построены по предположению индукции, и тогда на k -й итерации находится и искомым путь. $\square$



Рис. 2: Стивен Варшалл (1935–2006); Роберт Флойд (1936–2001).

Из леммы сразу следует правильность алгоритма: действительно, после n -й итерации внешнего цикла будут найдены все пути, проходящие через промежуточные вершины $\{1, \dots, n\}$, то есть совсем все пути.

1.3 Кратчайшие пути с весами

Если нужно не только найти транзитивное замыкание, но и научиться быстро находить кратчайшие пути между всеми парами вершин? Тогда нужно следить за длинами путей.

Более общая задача. Пусть для каждой дуги задан её *вес* $w_{i,j}$. Если дуги нет, можно положить $w_{i,j} = \infty$. Нужно найти пути наименьшего веса между всеми парами вершин.

Очевидного алгоритма здесь нет. Можно запустить алгоритм Дейкстры из каждой вершины, но это будет $|V|$ раз $|E| \log |V|$. Если рёбер много, время работы достигнет $|V|^3 \log |V|$.

Алгоритм Флойда–Варшалла (алгоритм 3): время $O(n^3)$.

Инвариант внешнего цикла подобен инварианту для алгоритма Варшалла.

Алгоритм 3 Алгоритм Флойда–Варшалла для нахождения всех кратчайших путей

Массив $d_{i,j}$, начальные значения: $d_{i,j} = \begin{cases} 0, & \text{если } i = j \\ w_{i,j}, & \text{если } (i, j) \in E \\ \infty, & \text{в противном случае} \end{cases}.$

```
1: for  $k = 1$  to  $n$  do
2:   for  $i = 1$  to  $n$  do
3:     for  $j = 1$  to  $n$  do
4:       if  $d_{i,j} > d_{i,k} + d_{k,j}$  then
5:          $d_{i,j} = d_{i,k} + d_{k,j}$ 
```

Лемма 3. После k -й итерации для каждой пары вершин (i, j) найден путь из i в j наименьшего веса среди путей, проходящих только через промежуточные вершины из множества $\{1, \dots, k\}$.

Чтобы построить сами кратчайшие пути, для каждой пары вершин запоминается вершина $\pi_{i,j}$ — следующая после i вершина на кратчайшем пути из i в j . Она будет обновляться одновременно с улучшением дуги.

1.4 Обобщение до полукольца

Из какой алгебраической структуры могут браться веса в алгоритме Флойда–Варшалла? Для этого надо присмотреться, что, собственно, алгоритм делает с этими весами. Используются две операции: сложение весов последовательно проходимых рёбер, и взятие минимума по всем возможным путям. От операций нужна ассоциативность и дистрибутивность, а также коммутативность минимума. Это *полукольцо*.

Определение 1. Полукольцо (*semiring*) — это кольцо, недосчитавшееся аксиомы о противоположном элементе относительно сложения.

Пример 1. Простейший пример полукольца — булево полукольцо $\{0, 1\}$, с конъюнкцией в качестве умножения и дизъюнкцией в качестве сложения: они ассоциативны и дистрибутивны, дизъюнкция коммутативна. Однако у единицы нет противоположного элемента, для которого выполнялось бы $1 \vee x = 0$ — и потому это не кольцо, а полукольцо.

В алгоритме Флойда–Варшалла используется полукольцо относительно следующих операций. В качестве умножения в полукольце используется *сложение* (!) чисел; в качестве сложения в полукольце — операция взятия минимума чисел. Тогда вес каждого пути — это произведение в полукольце весов его рёбер, а вес совокупности путей из u в v — сумма в полукольце весов всех простых путей.

Алгоритм Флойда–Варшалла работает во всяком полукольце. В частности, в булевом полукольце он вырождается в алгоритм Варшалла.

1.5 Достижимость в графе и умножение матриц

Достижимость в графе можно выразить через степени матрицы смежности графа. Матрица смежности — булева матрица $A \in \mathbb{B}^{n \times n}$, в которой элемент $a_{i,j}$ равен 1, если в графе есть дуга (i, j) , и равен 0, если дуги нет.

Произведение булевых матриц определяется в булевом полукольце. Иными словами, произведение булевых матриц $A \in \mathbb{B}^{m \times \ell}$ и $B \in \mathbb{B}^{\ell \times n}$ — это булева матрица $A \times B = C \in \mathbb{B}^{m \times n}$, со следующими значениями элементов.

$$c_{i,j} = \bigvee_{k=1}^{\ell} a_{i,k} \wedge b_{k,j}$$

Если $A \in \mathbb{B}^{n \times n}$ — матрица смежности графа, то A^ℓ — матрица достижимости по путям длины ровно ℓ . Пусть $I \in \mathbb{B}^{n \times n}$ — единичная матрица. Тогда $(A \vee I)^\ell$ — матрица достижимости по путям длины не более чем ℓ , а A^{n-1} — матрица достижимости. Обозначение: $A^* = (A \vee I)^{n-1}$ — рефлексивно-транзитивное замыкание.

Вычисляется возведением в квадрат $\log n$ раз. Если умножать матрицы напрямую, получится время $n^3 \log n$ — медленней, чем Варшалл. Но, во-первых, здесь векторные операции, их компьютеру выполнять удобнее, чем произвольный поиск по матрице смежности. Во-вторых, матрицы можно умножать существенно быстрее, чем за кубическое время!

2 Быстрое умножение матриц

Произведение двух матриц размера $n \times n$ по определению вычисляется за n^3 умножений и $n^2(n-1)$ сложений. Однако существуют алгоритмы умножения матриц, работающие быстрее, чем по определению, *быстрее очевидного* — и уже сама такая возможность удивительна.

2.1 Алгоритм Штрассена

Пусть $\mathcal{R}$ — кольцо, т.е., ассоциативные операции сложения и умножения, сложение коммутативно, дистрибутивность, нейтральные элементы, противоположный элемент относительно сложения (без противоположного элемента — *полукольцо*).

Сумма матриц $A \in \mathcal{R}^{m \times n}$, $B \in \mathcal{R}^{m \times n}$ — матрица $A + B = C \in \mathcal{R}^{m \times n}$.

$$c_{i,j} = a_{i,j} + b_{i,j}$$

Сложение ассоциативно, коммутативно, есть нейтральный и противоположный элементы.

Произведение матриц $A \in \mathcal{R}^{m \times \ell}$, $B \in \mathcal{R}^{\ell \times n}$ — матрица $A \times B = C \in \mathcal{R}^{m \times n}$.

$$c_{i,j} = \sum_{k=1}^{\ell} a_{i,k} \cdot b_{k,j}$$

Умножение ассоциативно, есть нейтральный элемент.

Утверждение 1. *Квадратные матрицы размера $n \times n$ над кольцом сами образуют кольцо.*

Произведение двух матриц 2×2 , согласно определению, выражается за 8 умножений и 4 сложения.

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \times \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}$$



Рис. 3: Фолькер Штрассен (род. 1936).

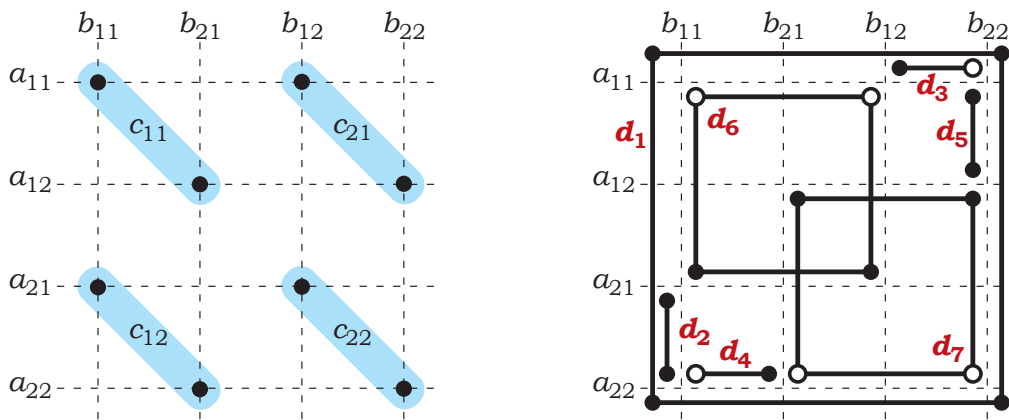


Рис. 4: (слева) Восемь произведений при умножении матриц 2×2 по определению; (справа) семь произведений при умножении по Штрассену.

Но есть удивительный способ вычисления этого же произведения за 7 умножений и 18 сложений.

Лемма 4 (Штрассен [1969]). Пусть $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$ и $B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$ — две матрицы размера 2×2 над кольцом $\mathcal{R}$. Тогда их произведение $A \times B = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$ можно вычислить за 7 умножений и 18 сложений в $\mathcal{R}$.

Доказательство. Восемь произведений, соответствующие определению произведения матриц, отмечены на рис. 4(левом), где горизонтальные пунктирные линии соответствуют четырём элементам A , вертикальные — четырём элементам B , а пересечение линий соответствует произведению элементов. Произведения разбиваются на 4 пары — суммы, соответствующие элементам искомой матрицы $A \times B$.

Чтобы вычислить эти же 4 суммы иначе, сперва вычисляются следующие 7 произведе-

ний.

$$\begin{aligned}
d_1 &= (a_{11} + a_{22})(b_{11} + b_{22}), \\
d_2 &= (a_{21} + a_{22})b_{11}, \\
d_3 &= a_{11}(b_{12} - b_{22}), \\
d_4 &= a_{22}(b_{21} - b_{11}), \\
d_5 &= (a_{11} + a_{12})b_{22}, \\
d_6 &= (a_{21} - a_{11})(b_{11} + b_{12}), \\
d_7 &= (a_{12} - a_{22})(b_{21} + b_{22}).
\end{aligned}$$

Они изображены на рис. 4(правом), где каждый заполненный круг соответствует произведению со знаком «плюс», а пустой — произведению, умноженному на -1 . Тогда искомое произведение $C = AB$ выражается так.

$$\begin{aligned}
c_{11} &= d_1 + d_4 + d_7 - d_5 \\
c_{12} &= d_3 + d_5 \\
c_{21} &= d_2 + d_4 \\
c_{22} &= d_1 + d_3 + d_6 - d_2
\end{aligned}$$

□

Если умножать надо числовые матрицы размера 2×2 , то такой способ вычисления их произведения окажется медленней, чем если считать согласно определению: действительно, легче проделать одно лишнее умножение, чем 14 лишних сложений. Смысл умножения матриц 2×2 за 7 умножений и много сложений в том, что эту формулу для произведения матриц можно применять рекурсивно, сводя умножение матриц размера $n \times n$ к умножению семи пар матриц вдвое меньшего размера — *семи*, а не восьми.

Сперва матрицы $n \times n$ представляются в виде блочных матриц, состоящих из четырёх подматриц размера $\frac{n}{2} \times \frac{n}{2}$ каждая.

$$\left(\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right) \times \left(\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right) = \left(\begin{array}{c|c} A_{11} \times B_{11} + A_{12} \times B_{21} & A_{11} \times B_{12} + A_{12} \times B_{22} \\ \hline A_{21} \times B_{11} + A_{22} \times B_{21} & A_{21} \times B_{12} + A_{22} \times B_{22} \end{array} \right)$$

Поскольку матрицы размера $\frac{n}{2} \times \frac{n}{2}$ над кольцом сами образуют кольцо, здесь записано произведение двух матриц размера 2×2 над этим кольцом. Согласно лемме 4, вычисление этого произведения сводится к умножению семи пар матриц вдвое меньшего размера, а также некоторому количеству сложений таких матриц. Наконец, для умножения матриц размера $\frac{n}{2} \times \frac{n}{2}$ рекурсивно используется этот же алгоритм.

Глубина рекурсии: $\log_2 n$. Задач размера n — одна. Задач размера $\frac{n}{2}$ — семь. Задач размера $\frac{n}{2^i}$ — всего 7^i . Для каждой задачи размера $\frac{n}{2^i}$, её внутреннее время работы, не считая рекурсивных вызовов, составляет $O((\frac{n}{2^i})^2)$ шагов. Всего:

$$\sum_{i=0}^{\log_2 n} 7^i \left(\frac{n}{2^i} \right)^2 = n^2 \sum_{i=0}^{\log_2 n} \left(\frac{7}{4} \right)^i = n^2 \frac{\left(\frac{7}{4} \right)^{1+\log_2 n} - 1}{\frac{7}{4} - 1} = O\left(n^2 \cdot \frac{7^{\log_2 n}}{4^{\log_2 n}} \right) = O\left(n^2 \cdot \frac{n^{\log_2 7}}{n^2} \right) = O(n^{\log_2 7})$$

Теорема 1 (Штрассен [1969]). Пусть $\mathcal{R}$ — кольцо, пусть $k \geq 0$, и пусть A и B — две матрицы $2^k \times 2^k$ над $\mathcal{R}$. Тогда произведение AB можно вычислить за 7^k умножений и $\Theta(7^k)$ сложений.

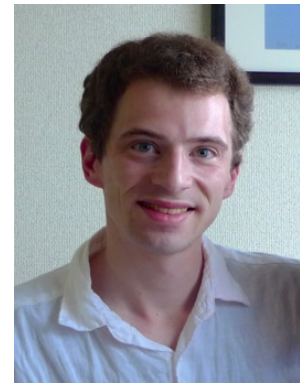


Рис. 5: Самуил Виноград (1936–2019), Дональд Копперсмит (род. ок. 1950), Франсуа Ле Галл.

Отсюда матрицы размера $n \times n$ можно перемножить за время $O(n^{\log_2 7})$.

Применяя метод Штрассена для умножения чисел с плавающей точкой, нужно иметь в виду, что тип данных `float` не образует кольца, и потому алгоритм не обязан вычислять правильные значения. На практике это может вылиться в накопление погрешности.

Позднее были получены разнообразные асимптотически более быстрые алгоритмы умножения матриц. Теоретически самый быстрый известный алгоритм — это метод Копперсмита–Винограда [1990] с улучшениями Ле Галла [2014], работающий за время $O(n^{2.373})$. Однако эти алгоритмы начинают работать быстрее Штрассена лишь для астрономических значений n , а для матриц, которые есть надежда когда-нибудь умножить, или хотя бы где-то записать, алгоритм Штрассена остаётся наилучшим.

Виноград [1971] показал методами линейной алгебры, что умножить матрицы размера 2×2 менее чем за 7 умножений нельзя, даже если использовать коммутативность умножения. Делались попытки улучшить алгоритм Штрассена, найдя способ умножить за малое число шагов квадратные матрицы немного большего размера, чем 2×2 , чтобы затем применить эти формулы рекурсивно. Если бы матрицы размера 3×3 удалось умножить за 21 умножение, или матрицы размера 4×4 — за 48 умножений, то получилось бы практически применимое улучшение алгоритма Штрассена. Однако найти такие формулы умножения по сей день не удаётся. Например, для матриц размера 3×3 Ладерман [1976] изобрёл формулы вычисления произведения за 23 умножения — но $\log_3 23 > \log_2 7$, и полученный алгоритм всё равно работает медленнее алгоритма Штрассена.

Задача 1. Рассмотреть формулы для умножения двух матриц размера 4×4 за 49 умножений — они получаются двухуровневым применением метода Штрассена. Можно ли что-то где-то переделать, чтобы обойтись 48 умножениями?

Нижних оценок времени умножения матриц произвольного размера нет, n^2 действий нужно просто для того, чтобы записать результат, и нельзя исключить, что матрицы на самом деле можно умножать за время $n^2(\log n)^{O(1)}$ — просто мы не знаем, как. Показатель степени при n во времени работы асимптотически наилучшего алгоритма умножения матриц обозначается через ω (омега), и известно лишь, что $2 \leq \omega < 2.373$.

2.2 Время работы рекурсивных алгоритмов

Для определения времени работы рекурсивных алгоритмов уже не раз использовалось одно и то же доказательство. Его стоит сформулировать в общем виде.

Теорема 2 (Основная теорема о времени работы рекурсивных алгоритмов). Пусть задача размера n решается путём разбиения её на a подзадач того же типа, размера $\frac{n}{b}$ каждая, и процесс разбиения, а также соединения результатов, занимает время $O(n^c)$.

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^c)$$

Тогда время работы алгоритма оценивается так.

- Если $c < \log_b a$, то $O(n^{\log_b a})$.
- Если $c = \log_b a$, то $O(n^c \log_b n)$.
- Если $c > \log_b a$, то $O(n^c)$.

Важно, что на каждом следующем уровне рекурсии размер подзадачи *делится на константу*. Если из него только лишь *вычитается константа* — скажем, если $T(n) = 2T(n-1) + O(n^c)$ — то время работы окажется экспоненциальным.

Доказательство. В дереве рекурсии $\lceil \log_b n \rceil$ уровней, на 0-м — 1 подзадача размера n , расчёты занимают время n^c ; на 1-м — a подзадач размера $\lceil \frac{n}{b} \rceil$, расчёты для каждой занимают время $\lceil \frac{n}{b} \rceil^c$; и далее, на каждом i -м уровне — a^i подзадач размера $\lceil \frac{n}{b^i} \rceil$ каждая, расчёты для каждой занимают время $\lceil \frac{n}{b^i} \rceil^c$. Общее время работы — сумма по всем уровням рекурсии.

$$T(n) = \sum_{i=0}^{\lceil \log_b n \rceil} a^i \left(\frac{n}{b^i}\right)^c = n^c \sum_{i=0}^{\lceil \log_b n \rceil} \left(\frac{a}{b^c}\right)^i$$

Если $a = b^c$, то сумма геометрической прогрессии — $\log_b n$, и отсюда выходит $O(n^c \log_b n)$. Если $a < b^c$, то сумма ограничена сверху константой, отсюда $O(n^c)$. Если же $a > b^c$, то это возрастающая геометрическая прогрессия, и её сумма имеет следующий вид.

$$n^c \cdot \sum_{i=0}^{\lceil \log_b n \rceil} \left(\frac{a}{b^c}\right)^i = n^c \cdot \frac{\left(\frac{a}{b^c}\right)^{\log_b n} - 1}{\frac{a}{b^c} - 1} = O\left(n^c \cdot \frac{a^{\log_b n}}{b^{c \log_b n}}\right) = O\left(n^c \cdot \frac{n^{\log_b a}}{n^c}\right) = O(n^{\log_b a})$$

□

Пример 2. Сортировка слиянием: $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$, т.е. $a = 2$, $b = 2$, $c = 1$. Тогда $\log_b a = c$, и, по теореме, $T(n) = O(n^c \log_b n) = O(n \log n)$.

Пример 3. Быстрое умножение Карацубы: $T(n) = 3T\left(\frac{n}{2}\right) + O(n)$, т.е. $a = 3$, $b = 2$, $c = 1$. Тогда $\log_b a > c$, и, по теореме, $T(n) = O(n^{\log_b a}) = O(n^{\log_2 3})$.

Пример 4. Быстрое умножение матриц Штрассена: $T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$, т.е. $a = 7$, $b = 2$, $c = 2$. Тогда $\log_b a > c$, и, по теореме, $T(n) = O(n^{\log_b a}) = O(n^{\log_2 7})$.



Рис. 6: Владимир Арлазаров (род. 1939), Ефим Диниц (род. 1949), Михаил Кронрод, Игорь Фараджев (1939 или 1940–2020).

2.3 Умножение булевых матриц через числовые

Булевы матрицы определены над *булевым полукольцом*, а не над кольцом, поскольку нет противоположного элемента для сложения. Поэтому алгоритм Штрассена сам по себе неприменим. Однако можно умножить булевы матрицы через числовые следующим образом.

Сперва булевы матрицы записываются как числовые матрицы с элементами 0 и 1, и затем они перемножаются в кольце вычетов по модулю $n + 1$ — или по любому удобному модулю, превосходящему n . Тогда вместо дизъюнкции конъюнкций $\bigvee_{k=1}^{\ell} a_{i,k} \wedge b_{k,j}$ будет вычислена сумма произведений $\sum_{k=1}^{\ell} a_{i,k} \cdot b_{k,j}$ по модулю $n+1$. Сама по себе сумма $\sum_{k=1}^{\ell} a_{i,k} \cdot b_{k,j}$ в точности равна количеству истинных конъюнкций в дизъюнкции конъюнкций, и потому она лежит в диапазоне от 0 до n — откуда следует, что по модулю $n + 1$ она вычислится точно. Чтобы узнать значение дизъюнкции конъюнкций, достаточно будет проверить вычисленную сумму произведений на неравенство нулю.

3 Метод четырёх русских

Первый чисто комбинаторный метод умножения булевых матриц за время $o(n^3)$ изобрели Арлазаров, Диниц, Кронрод и Фараджев [1970], и в честь первооткрывателей он носит имя *метода четырёх русских* (Method of Four Russians).

Пусть A и B — две булевы матрицы размера $n \times n$, цель — вычислить их произведение $C = AB$. Пусть k — число, немного меньшее, чем $\log_2 n$, и пусть n делится на k ; если не делится, то n можно немного увеличить, чтобы делилось. Каждая строка матрицы A делится на $\frac{n}{k}$ векторов размера $1 \times k$, называемых *кусочками*. Кусочки в i -й строке обозначаются через $A_{i,1}, \dots, A_{i,\frac{n}{k}} \in \mathbb{B}^{1 \times k}$. Матрица B разделяется на $\frac{n}{k}$ подматриц размера $k \times n$, называемых *полосами* и обозначаемых через $B_1, \dots, B_{\frac{n}{k}} \in \mathbb{B}^{k \times n}$. Тогда всякая i -я строка C , обозначаемая через $C_i \in \mathbb{B}^{1 \times n}$, представима в виде следующей поэлементной дизъюнкции строк.

$$C_i = \bigvee_{r=1}^{\frac{n}{k}} A_{i,r} B_r$$

Произведение кусочка $A_{i,r}$ на соответствующую полосу B_r , показанное на рис. 7 — это строка размера $1 \times n$, и знак дизъюнкции в формуле для C_i означает поразрядную дизъюнцию таких строк¹.

¹Почему это верно? По определению произведения матриц, $c_{i,j} = 1$, если $a_{i,\ell} = 1$ и $b_{\ell,j} = 1$ для какого-то ℓ . Пусть r — номер кусочка, в который попал элемент $a_{i,\ell}$, и пусть t — позиция внутри этого кусочка. Тогда

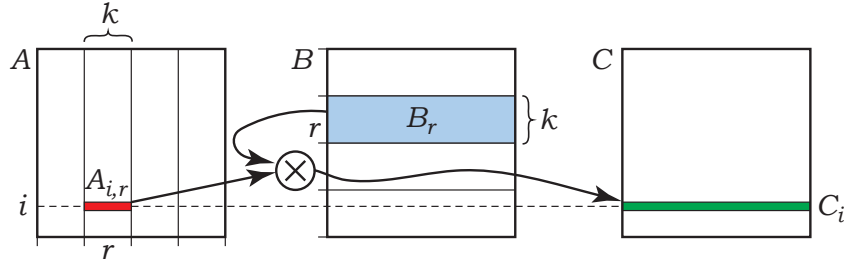


Рис. 7: Метод четырёх русских: произведение кусочка $A_{i,r}$ размера $1 \times k$ на полосу B_r размера $k \times n$, вносящее свой вклад в i -ю строку матрицы C .

Эта формула для вычисления произведения булевых матриц математически ничем не отличается от вычисления по определению: следуя ей, будут вычислены те же самые n^3 конъюнкций и $(n-1)n^2$ дизъюнкций. Чисто программистские улучшения в ней есть: вычислять поразрядные дизъюнкции строк матрицы на компьютерах очень удобно, такие однотипные векторные операции будут выполняться куда быстрее, чем если ковыряться в отдельных битах.

Но главная идея метода четырёх русских в другом. Так как значение k невелико, возможных кусочков всего 2^k , и одни и те же кусочки будут встречаться повторно. Поэтому в ходе работы алгоритма одинаковые кусочки будут то и дело умножаться на одну и ту же полосу. Вместо того, чтобы всякий раз умножать их заново, стоит заранее вычислить произведения *всех возможных кусочков* размера $1 \times k$ со всеми полосами матрицы B . Для каждого кусочка $(x_1, \dots, x_k) \in \mathbb{B}^k$ и для каждой полосы B_r , алгоритм вычисляет следующее произведение вектора на матрицу.

$$D_r[x_1, \dots, x_k] = (x_1, \dots, x_k) \cdot B_r$$

Получившаяся строка размера $1 \times n$ сохраняется в памяти. Это произведение можно вычислить с помощью поразрядной дизъюнкции всех строк B_r , соответствующих единичным битам кусочка; такая операция эффективно реализуется на компьютерах.

Построив таблицу, алгоритм вычисляет всякую i -ю строку матрицы C в виде следующей поразрядной дизъюнкции строк: каждое произведение $A_{i,r}B_r$ берётся из таблицы, проиндексированной k битами кусочка $A_{i,r}$ и номером полосы r .

$$C_i = \bigvee_{r=1}^{\frac{n}{k}} D_r[A_{i,r}],$$

Осталось подсчитать общее число операций над битами, вычисляемых алгоритмами. На этапе построения таблицы рассматриваются 2^k возможных значений кусочков, матрица B состоит из $\frac{n}{k}$ полосок, и всякое произведение требует kn битовых операций. Поэтому выполняется $2^k \cdot \frac{n}{k} \cdot kn = 2^k n^2$ операций над битами. Далее, каждой из n строк матрицы C вычисляется поразрядной дизъюнкцией $\frac{n}{k}$ строк длины n битов каждая — всего $\frac{n^3}{k}$ операций. Пусть $k = \log_2 n - \log_2 \log_2 n$. Тогда общее число операций подсчитывается так.

$$2^k n^2 + \frac{n^3}{k} = \frac{n^3}{\log_2 n} + \frac{n^3}{\log_2 n - \log_2 \log_2 n} = O\left(\frac{n^3}{\log n}\right)$$

При реализации на современном компьютере алгоритм может хранить много битов в машинном слове и использовать поэлементную дизъюнкцию машинных слов, чтобы произвести много операций над битами за одну инструкцию процессора.

в произведении $A_{i,r}B_r$ будет вычислена конъюнкция $a_{i,\ell} \wedge b_{\ell,j}$, и она попадёт в j -ю позицию строки C_i .

На практике метод четырёх русских следует использовать для матриц, начиная примерно с 16×16 и заканчивая примерно 8192×8192 . Для матриц большего размера более эффективным оказывается метод Штрассена в кольце по модулю $n + 1$.

Список литературы

- [1970] В. Л. Арлазаров, Е. А. Диниц, М. А. Кронрод, И. А. Фараджев, “Об экономном построении транзитивного замыкания ориентированного графа”, *Доклады Академии наук СССР*, 194:3 (1970), 487–488.
- [1990] D. Coppersmith, S. Winograd, “Matrix multiplication via arithmetic progressions”, *Journal of Symbolic Computation*, 9:3 (1990), 251–280.
- [1976] J. D. Laderman, “A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications”, *Bulletin of the AMS*, 82:1 (1976), 126–128.
- [2014] F. Le Gall, “Powers of tensors and fast matrix multiplication”, ISSAC 2014, 296–303.
- [1969] V. Strassen, “Gaussian elimination is not optimal”, *Numerische Mathematik*, 13 (1969), 354–356.
- [1962] S. Warshall, “A theorem on Boolean matrices”, *Journal of the ACM*, 9:1 (1962), 11–12.
- [1971] S. Winograd, “On multiplication of 2×2 matrices”, *Linear Algebra and its Applications*, 4:4 (1971), 381–388.